Sieci Neuronowe
Laboratorium 02
Wstęp do Pythona i podstaw sieci neuronowych

Andrzej Kordecki

Zakład Teorii Maszyn i Robotów
Instytut Techniki Lotniczej i Mechaniki Stosowanej
Wydział Mechaniczny Energetyki i Lotnictwa
Politechnika Warszawska
2026

Sztuczny neuron

Celem drugiego laboratorium jest rozwijanie umiejętności programowania w Pythonie (NumPy, matplotlib, klasy, funkcje, słowniki) oraz implementacja podstawowych operacji sztucznego neuronu: wyznaczania potencjału, funkcji aktywacji i ich pochodnych, a także funkcji strat. Przygotowujemy natomiast kod, który zostanie rozbudowany w kolejnych trzech laboratoriach do wielowarstwowej sieci neuronowej oraz algorytmu propagacji wstecznej błędu (backpropagation).

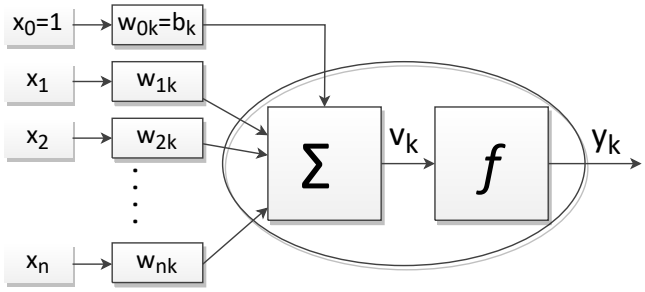
Zajęcia mają charakter wprowadzający i nie są jeszcze oceniane punktowo. Zadania należy wykonać i sprawdzić, ponieważ ich kod powininen być wykorzystywany w kolejnych, ocenianych laboratoriach. Korzystamy z Pythona, NumPy i matplotlib (np. w Google Colab). Nie korzystamy z gotowych warstw, funkcji strat ani mechanizmu automatycznego różniczkowania PyTorch/TensorFlow.

1 Sztuczny neuron i jego reprezentacja jako warstwa sieci neuronowej

Sztuczny neuron łączy wartości wejściowe z odpowiadającymi im wagami, a następnie przekształca otrzymany potencjał za pomocą funkcji aktywacji. W wyniku działania sztucznego neuronu otrzymujemy wartość wyjściową, którą można dalej przekazać jako wejście do kolejnych neuronów w sieci neuronowej. Dla neuronu k:

$$v_k = \sum_{i=1}^{n} w_{ik} x_i + b_k, \qquad y_k = \phi(v_k). \tag{1}$$

Symbole x_i oznaczają wejścia, w_{ik} – wagi, b_k – przesunięcie (*bias* lub obciążenie), v_k – efektywne wejście funkcji aktywacji (potencjał neuronu), a y_k – jego wyjście.



W zadaniach obecnego laboratorium pomijamy b_k, przyjmując $b_k = 0$. W dalszych laboratoriach możliwe będzie wprowadzenie tego parametru. Warto już teraz pomyśleć o reprezentacji wielu neuronów naraz. Dla pojedynczego wejścia warstwy (wektor $\mathbf{x}$) i macierzy wag W:

$$\mathbf{v} = W\mathbf{x} + \mathbf{b}, \qquad \mathbf{y} = \phi(\mathbf{v}), \tag{2}$$

przy czym W ma wymiary $(n_{\text{neurons}}, n_{\text{inputs}})$, a $\mathbf{x}$ – $(n_{\text{inputs}},)$. Funkcja aktywacji działa elementowo. W laboratorium zaczniemy od pojedynczego neuronu, ale zachowamy wektorową reprezentację potencjału i wyjścia, także gdy wektor zawiera jeden element.

2 Funkcje aktywacji i ich pochodne

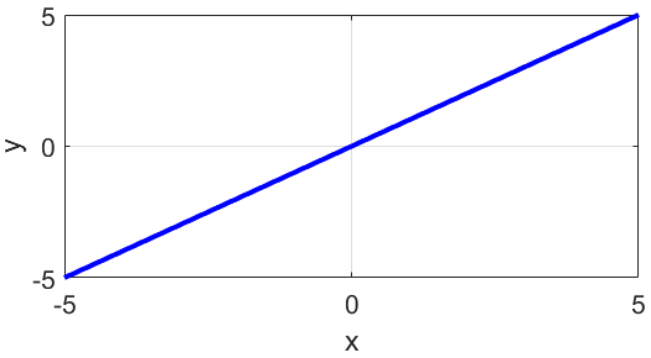
Nieliniowe funkcje aktywacji umożliwiają sieci złożonej z wielu warstw odwzorowywanie zależności innych niż liniowe. Pochodne funkcji będą potrzebne w algorytmie propagacji wstecznej błędu. We wszystkich wzorach v oznacza potencjał neuronu, a $\phi'(v)$ – pochodną względem tego potencjału.

Funkcja tożsamościowa (linear)

Wzór na funkcję i jej pochodną:

$$\phi(v) = v, \qquad \phi'(v) = 1. \tag{3}$$

Można rozważać ogólniejszą funkcję liniową av, której pochodna wynosi a, lecz w zadaniach stosujemy funkcję tożsamościową.

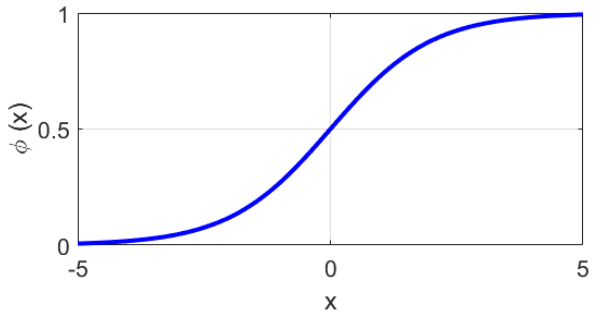


Funkcja logistyczna (sigmoid)

Wzór na funkcję i jej pochodną:

$$\phi(v) = \frac{1}{1+e^{-v}}, \qquad \phi'(v) = \phi(v)\big(1-\phi(v)\big). \tag{4}$$

Wyjście funkcji należy do przedziału $(0, 1)$. Pochodną można obliczyć z wyjścia funkcji aktywacji, bez ponownego liczenia wykładnika.

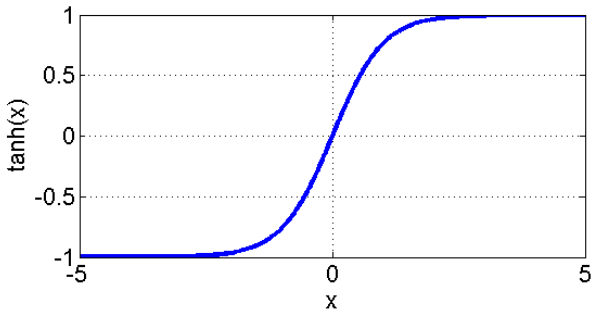


Tangens hiperboliczny (tanh)

Wzór na funkcję i jej pochodną:

$$\phi(v) = \tanh(v) = \frac{e^{v}-e^{-v}}{e^{v}+e^{-v}}, \qquad \phi'(v) = 1-\phi(v)^2. \tag{5}$$

Wyjście należy do przedziału $(-1, 1)$. Również tutaj pochodną można wyznaczyć na podstawie obliczonego wyjścia.

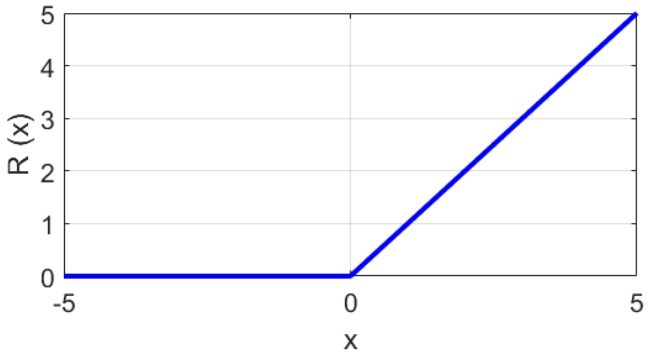


ReLU (Rectified Linear Unit)

Wzór na funkcję i jej pochodną:

$$\phi(v) = \max(0, v), \qquad \phi'(v) = \begin{cases} 1, & v > 0, \\ 0, & v \leq 0. \end{cases} \tag{6}$$

W punkcie $v = 0$ funkcja nie jest różniczkowalna. Na potrzeby implementacji przyjmujemy tam wartość pochodnej równą 0 (często stosowana konwencja w bibliotekach).



3 Funkcje strat

Funkcja straty określa niezgodność predykcji y z wartością oczekiwaną (etykietą) t. W uczeniu sieci jej pochodna stanowi punkt wyjścia do obliczania gradientów parametrów. W zadaniach laboratoryjnych rozważamy dwie funkcje; pozostałe poznane na wykładzie, np. MAE i stratę Hubera, mogą służyć jako materiał dodatkowy. Implementowana w programie funkcja strat odnosi się do pojedynczego próbki, a nie sumowania, które będzie wykonane w funkcji głównej programu w późniejszych laboratoriach.

Błąd kwadratowy

Dla pojedynczej wartości wyjściowej przyjmujemy:

$$\ell_{\text{sq}}(t, y) = \frac{1}{2}(y - t)^2, \qquad \frac{\partial \ell_{\text{sq}}}{\partial y} = y - t. \tag{7}$$

Czynnik $\frac{1}{2}$ upraszcza pochodną. To połowa błędu kwadratowego, a nie standardowa definicja MSE. Dla N niezależnych elementów można zdefiniować średnią stratę

$$L_{\text{sq}} = \frac{1}{N} \sum_{i=1}^{N} \ell_{\text{sq}}(t_i, y_i), \qquad \frac{\partial L_{\text{sq}}}{\partial y_i} = \frac{y_i - t_i}{N}. \tag{8}$$

Standardowe MSE nie zawiera czynnika $\frac{1}{2}$. W zadaniach niżej używamy nazwy `mse` jako uproszczonej nazwy interfejsu dla połowy średniego błędu kwadratowego ze względu na prostszy wzór. Należy pamiętać o tej konwencji podczas porównań z bibliotekami.

Binarna entropia krzyżowa (binary cross-entropy, BCE)

Dla etykiety $t \in \{0, 1\}$ i predykcji interpretowanej jako prawdopodobieństwo $y \in (0, 1)$:

$$\ell_{\mathrm{BCE}}(t, y) = -\big[t \ln(y) + (1 - t) \ln(1 - y)\big], \tag{9}$$

$$\frac{\partial \ell_{\mathrm{BCE}}}{\partial y} = -\frac{t}{y} + \frac{1 - t}{1 - y}. \tag{10}$$

Dla zbioru wyników również można przyjąć średnią po N elementach (wtedy pochodne dzielimy przez N). Ponieważ $\ln(0)$ jest nieokreślony, w implementacji operującej na prawdopodobieństwach należy ograniczyć y do $[\varepsilon, 1-\varepsilon]$ (np. $\varepsilon = 10^{-7}$). Przy czym taka skrajna sytuacja, gdzie liczmy logarytm dla zera ma małą szanse zaistnienia.

Zadania

Uwagi do zadań

Ważna zasada projektowa. Tworzony kod będzie rozwijany podczas następnych trzech laboratoriów. Przyjmujemy wspólny interfejs oraz spójne nazwy; można rozbudować implementację, ale późniejsze zadania powinny nadal korzystać z tych samych funkcji. Przy operacjach matematycznych używaj `numpy.ndarray`, unikając zbędnych pętli i ręcznego przeliczania każdego neuronu.

Zadanie 1

Stwórz funkcję dane wejściowe do wyznaczania funkcji aktywacji oraz odpowiednią strukturę jednego neuronu. Wejściem funkcji są:

- inputs - lista wartości wejściowych neuronu. Każdy element listy odpowiada jednemu wejściu neuronu (próbce) i powinny być skalarem dla pojedynczej próbki. W kolejnych laboratoriach lista może składać się z próbek w postaci tablic NumPy zawierającą wartości wejścia sieci neuronowej.
- weights - lista wag jednego neuronu, po jednej dla każdego wejścia. Wagi są początkowo generowane losowo. Dla pojedynczej próbki obliczony potencjał neuronu jest skalarem. W przypadku jednoczesnego przetwarzania wielu próbek potencjał może być wektorem zawierającym wynik dla każdej próbki. W kolejnych laboratoriach będzie zastąpiony funkcją wyznaczajacą macierz wag, czyli zapisem wag dla każdego nauronu warswy.

Początkowe wagi neuronu powinny zostać wygenerowane w sposób losowy z wykorzystaniem biblioteki numpy zgodnie z następującym kodem:

```
import numpy as np

np.random.seed(42)

input_size = 10 # lub inna zadana liczba

inputs = [np.random.randn() for i in range(input_size)]
weights = [np.random.randn() for i in range(input_size)]
```

Wejściem nowej funkcji aktywacji powinna być słownik z odpowiednio dobraną strukturą kluczy: określają wartości wejściowe neuronu i w późniejszych laboratoriach wartości wejściowe warstwy sieci neuronowej, które są wyjściami poprzedniej warstwy. Struktura danych wejściowych powinna być następująca:

```
layer = {
  'activation_potential': np.matmul(weights, inputs),
  'output': None
}
print(layer)
```

Wynik potencjału pojedynczego neuronu jest matematycznie skalarem, ale w słowniku przechowujemy tablicę `shape == (1,)`. Dzięki temu w kolejnych zadaniach będzie można bez zmiany interfejsu przechowywać potencjały całej warstwy. Sprawdź kształty wektorów wejść i wag oraz tablicy potencjałów w słowniku. Porównaj wynik z obliczeniem przez sumę iloczynów elementów:

Zadanie 2

Stwórz z wykorzystaniem kodu napisanego w poprzednim zadaniu funkcję wyznaczającą wyjście neuronu dla zadanej funkcji aktywacji lub wyjście pochodnej funkcji aktywacji. W ramach zadania napisz cztery nowe funkcje, które będą wyznaczać wyjścia następujących funkcji aktywacji:

- "linear funkcja tożsamości (liniowa),
- "sigmoid funkcja logistyczną,
- "tanh funkcja tangensu hiperbolicznego,
- "ReLu funkcja ReLu.

Określenie w ten sposób wejścia wpływa na strukturę funkcji wyznaczająca wartość wyjściową funkcji aktywacji. Struktura każdej funkcji powinna być następująca na przykładzie liniowej funkcji aktywacji:

```
import numpy as np
import sys

def linear(activation_potential, output, derivative=False):
  if not derivative:
    return activation_potential
  else:
    return np.ones_like(activation_potential)

# Funkcje mozna zapisac znacznie prosciej
# def linear(activation_potential, output, derivative=False):
#   return activation_potential if not derivative else np.ones_like(activation_potential)

activation_potential = layer['activation_potential']

# Output bwezie potrzebne w fazie wstec algorytmu treningu sieci,
# gdzie wyjscie neuronu bedzie wczejsciej wyznaczone
layer['output'] = linear(activation_potential, 0)
linear(layer)
```

Argumentem funkcji aktywacji jest parametr "derivative", który określa, czy wyjściem funkcji jest funkcja aktywacji lub pochodna funkcji aktywacji dla danego wejścia.

Zadanie 3

Stwórz funkcję do wizualizacji zmian przebiegu funkcji aktywacji dla zadanego zakresu zmian wartości potencjału neuronu 'ap' (nie wejścia - inputs, a potencjału). Wyjściem funkcji powinien być wykres stworzony za pomocą biblioteki matplotlib. Na wykresie zmienna oś X powinna pokazywać potencjał aktywacji, a oś Y wyjście funkcji aktywacji. Przykładowy zakres zmian wartości potencjału neuronu:

```
layer = {
  'activation_potential': np.arange(-4.0, 4.0, 0.1),
  'output': None
}

linear(layer)
```

Wykresy możemy łatwo narysować z wykorzystaniem biblioteki "matplotlib.pyplot":

```
import matplotlib.pyplot as plt
plt.figure
plt.plot(layer['activation_potential'], linear(layer['activation_potential'], derivative=False))
plt.show()
```

Sprawdź przebieg wyjścia funkcji aktywacji, czy jest zgodny z oczekiwanym rezultatem jaki możemy zobaczyć w pierwszej części instrukcji.

Zadanie 4

Stwórz klasę "Activation_fcn", która będzie będzie zawierała wszystkie funkcje aktywacji z poprzednich zadań. Klasa "Activation_fcn"powinna zawierać istotne funkcje przedstawione w kodzie poniżej lub ich odpowiedniki. Klasa składa się z dwóch funkcji: konstruktora definiującego nazwy funkcji aktywacji w ramach klasy "Activation_fcn"oraz funkcji "output", która powinna umożliwiać wyznaczenie wyjście dowolnej funkcji aktywacji spośród funkcji zdefiniowanych w "self.functions".

Dodaj kod odpowiedzialny za wyznaczania wyjść poszczególnych funkcji aktywacji z poprzednich zadań do wstępnie napisanej klasy. Następnie odpowiednio zmodyfikuj kod programu, aby możliwe było wyznaczenie wyjście funkcji aktywacji za pomocą funkcji 'output' dla obiektu klasy "Activation_fcn". Przyjmij jak w zadaniu 1 wartości wejściowe (inputs) i wagi (weights).

```
import sys

class Activation_fcn:
    def __init__(self):
    self.functions = {
      'linear': self.linear,
```

```
      'sigmoid': self.logistic,
      'logistic': self.logistic,
      'tanh': self.tanh,
      'relu': self.relu
    }

  def output(self, layer, name, derivative=False):
    if name in self.functions:
      return self.functions[name](layer['activation_potential'], layer['output'], derivative)
    else:
      sys.exit(f"Error: Activation function '{name}' not found.")

  # Identity
  def linear(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  # Logistic (sigmoid)
  def logistic(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  # Hyperbolic tangent
  def tanh(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  #  ReLU
  def relu(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia
```

Po uzupełnieniu kodu funkcji aktywacji możemy sprawdzić jej działanie. Przykład dla liniowej funkcji aktywacji:

```
afcn = Activation_fcn()
afcn.output(layer, 'linear')
```

Zadanie 5

Stwórz wybrane funkcje strat stosowane w nauce sieci neuronwej, których argumentem jest "expected"i "output". Parametr "expected"określa oczekiwaną wartość wyjściową sieci neuronowej określoną na podstawie zbioru danych. Parametr "output"określa wartość wyjściową wyznaczoną przez sieć neuronową. W ramach zadania napisz dwie nowe funkcje, które będą wyznaczać wyjścia następujących funkcji strat:

- "mse błąd średnio kwadratowy,
- "bc_entropy funkcje binarnej entropii krzyżowej.

Struktura funkcji powinna być analogiczna do struktury dla funkcji aktywacji:

```
def mse(self, expected, outputs, derivative=False):
  if not derivative:
    return 0.5 * np.power(expected - outputs, 2)
  else:
    return -(expected - outputs)
```

Następnie stwórz klasę "Loss_fcn", która będzie będzie zawierała wszystkie funkcje strat ozadanej strukturze

```
class Loss_fcn:
  def __init__(self):
    self.functions = {
      'mse': self.mse,
      'binary_cross_entropy': self.binary_cross_entropy
    }

  # Loss/error value calculated for all input data sample
  def output(self, name, expected, outputs, derivative):
    if name in self.functions:
      return self.functions[name](expected, outputs, derivative)
    else:
      sys.exit(f"Error: Loss function '{name}' not found.")

  # Mean Square Error loss function
  def mse(self, expected, outputs, derivative=False):
    pass # do uzupelnienia
```

```
# Cross-entropy loss function
def binary_cross_entropy(self, expected, outputs, derivative=False):
  pass # do uzupelnienia
```

Zadanie 6

Stwórz funkcję do wizualizacji zmian przebiegu funkcji strat dla zadanej przez użytkownika różnicy wartości wyjściowej oraz zadanej przez użytkownika wartości "wyznaczonej"przez model. Dobierz obie zadane wartości tak, aby dobrze wykonywały wizualizację wyjść. Pamiętaj, że obie zaimplementowane wcześniej funkcje operują na innych zakresach wartości oraz wartość funkcji strat powinna być większe im większa jest różnica pomiędzy wartości zadaną i "wyznaczoną"przez model. Wyjściem funkcji powinien być wykres stworzony za pomocą biblioteki matplotlib. Na wykresie zmienna oś X powinna pokazywać różnicę pomiędzy wartości zadaną i "wyznaczoną"przez model, a oś Y wyjście funkcji strat.